

第 11 章：Model-based RL（Model-based Reinforcement Learning，基于模型的强化学习）、Offline RL（离线强化学习）、Imitation Learning 与 RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）

本章符号说明

符号/缩写	English	中文	含义
RL	Reinforcement Learning	强化学习	通过交互或数据学习策略以最大化长期 reward。
Offline RL	Offline Reinforcement Learning	离线强化学习	只用固定数据集学习策略，不继续和环境交互。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model，再优化 policy。
$P(s' s, a)$	Transition Probability	状态转移概率	环境 dynamics 的概率形式。
$R(s, a)$	Reward Function	奖励函数	环境或 reward model 给出的奖励。
D	Dataset	数据集	offline RL 或 imitation learning 使用的固定样本集合。
$f_{\phi}(s, a)$	Dynamics Model	动力学模型	参数 ϕ 下预测下一状态的模型。
BC	Behavior Cloning	行为克隆	把专家示范当监督学习来模仿。
IRL	Inverse Reinforcement Learning	逆强化学习	从专家行为反推出 reward function。
SFT	Supervised Fine-Tuning	监督微调	RLHF 中用人工示范数据微调语言模型。
RM	Reward Model	奖励模型	根据偏好数据预测 response 质量的模型。
DPO	Direct Preference Optimization	直接偏好优化	不显式跑 PPO 的偏好优化方法。

符号/缩写	English	中文	含义
KL	Kullback-Leibler Divergence	KL 散度	RLHF 中常用来限制 policy 偏离 reference model。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	文中作为 model-free 或 RLHF 优化阶段的代表算法。
BCQ	Batch-Constrained deep Q-learning	批约束深度 Q 学习	offline RL 代表算法之一。
BEAR	Bootstrapping Error Accumulation Reduction	bootstrap 误差累积降低	offline RL 代表算法之一。
CQL	Conservative Q-Learning	保守 Q 学习	通过保守估计 Q 值处理 OOD action 的 offline RL 方法。
IQL	Implicit Q-Learning	隐式 Q 学习	不显式查询 OOD action 的 offline RL 方法。
OOD	Out-of-Distribution	分布外	数据集中缺少覆盖的状态或动作。

本章目标

本章补充面试中越来越常见的进阶主题。你需要掌握：

- model-free vs model-based RL。
- offline RL 的核心难点。
- imitation learning 和 behavior cloning。
- inverse reinforcement learning。
- RLHF 的基本流程。

本章主线

前 10 章主要讨论在线 model-free 学习。本章换成四类现实约束：能不能学或使用环境模型，能不能继续交互，能不能从专家示范学，能不能用人类偏好定义 reward。Model-based RL、Offline RL、Imitation Learning、RLHF 分别对应这些约束，是面试里从经典 RL 扩展到真实系统和大模型对齐的关键桥梁。

四类进阶主题的边界

这一章主题多，先用边界表把它们分开：

主题	核心问题	数据来源	主要风险
Model-based RL	能不能学一个环境模型来 planning	在线交互数据或模拟器	model bias 和长 horizon 误差累积

主题	核心问题	数据来源	主要风险
Offline RL	能不能只用历史数据学新策略	固定日志数据集	OOD action 和 extrapolation error
Imitation Learning	能不能不设计 reward，直接模仿专家	专家状态-动作数据	compounding error
RLHF	能不能用人类偏好构造 reward	prompt-response 偏好比较	reward hacking、偏好噪声、KL 约束

它们共同点是：真实交互昂贵、危险或不充分，因此不能只靠“在线试错 + model-free 更新”解决。

Model-free vs Model-based

Model-free RL 不显式学习环境模型，直接学习 value 或 policy：

- Q-learning。
- DQN。
- PPO。
- SAC。

Model-based RL 显式学习或使用环境模型：

$$P(s' \mid s, a), R(s, a)$$

然后基于模型进行 planning 或生成模拟数据。

Model-based RL 的流程

典型流程：

1. 收集环境交互数据。
2. 学习 dynamics model：

$$s_{t+1} \approx f_{\phi}(s_t, a_t)$$

3. 用模型进行 planning 或 rollout。
4. 根据真实环境反馈继续修正模型。

优点：

- sample efficiency 可能更高。
- 可以通过模型进行规划。
- 在真实交互昂贵的场景中有吸引力。

缺点：

- 模型误差会累积。

- 长 horizon rollout 容易偏离真实环境。
- 高维复杂环境中 dynamics 很难学准。

面试表达：

Model-based RL 的核心优势是样本效率，但主要问题是 model bias。学到的 dynamics model 有误差，规划或 rollout 会放大这些误差，尤其在长时序任务中更明显。

Offline RL

Offline RL，也叫 batch RL，目标是在不继续和环境交互的情况下，只用固定数据集学习策略。

数据集：

$$D = \{(s, a, r, s')\}$$

这些数据来自某个 behavior policy，可能是人类、旧系统或混合策略。

与 online RL 的区别：

- online RL 可以探索并收集新数据。
- offline RL 只能使用已有数据。

Offline RL 的难点

核心问题：distribution shift。

学习到的新策略可能选择数据集中很少出现甚至没出现过的动作。此时 Q 函数对这些 out-of-distribution actions 的估计不可靠。

如果 Q 函数错误高估了某些 OOD 动作，policy improvement 会进一步偏向这些动作，导致性能崩溃。

这也叫 extrapolation error。

面试表达：

Offline RL 难在不能通过真实环境纠错。策略可能选择数据分布之外的动作，而 critic 对这些动作的 Q 值没有可靠监督，容易产生过估计并被 policy 利用。

Offline RL 的常见思路

常见方法会让策略不要偏离数据分布太远，或让 Q 值对 OOD 动作更保守。

几类思路：

- Behavior regularization：限制 learned policy 接近 behavior policy。
- Conservative value estimation：压低 OOD action 的 Q 值。
- Importance weighting：修正策略分布差异。
- Sequence modeling：把决策问题转成条件序列建模，例如 Decision Transformer。

代表算法了解即可：

- BCQ。
- BEAR。
- CQL。
- IQL。
- Decision Transformer。

Imitation Learning

Imitation Learning 从专家数据中学习策略。

专家数据：

$$(s, a_{expert})$$

目标：让 agent 模仿专家动作。

Behavior Cloning

Behavior Cloning，简称 BC，是最简单的 imitation learning。

把专家数据当监督学习：

$$\max_{\theta} \log \pi_{\theta}(a_{expert} | s)$$

或者最小化：

$$L(\theta) = -\log \pi_{\theta}(a_{expert} | s)$$

优点：

- 简单。
- 稳定。
- 不需要 reward。
- 训练像监督学习。

缺点：

- distribution shift。
- compounding error。
- 遇到专家数据中没覆盖的状态时容易出错。

为什么会 compounding error?

训练时模型只见过专家状态分布。部署时一旦模型犯小错，进入专家没覆盖的状态，就更容易继续犯错，误差逐步累积。

DAgger

DAgger 是 Dataset Aggregation。

核心思路：

1. 用当前策略和环境交互。
2. 让专家给当前策略访问到的状态标注正确动作。
3. 把新数据加入数据集。
4. 重新训练策略。

DAgger 通过收集 learner 自己会遇到的状态，缓解 behavior cloning 的分布偏移。

Inverse Reinforcement Learning

IRL 的目标不是直接模仿动作，而是从专家行为中反推 reward function。

直觉：

专家之所以这么行动，是因为它在优化某个隐藏 reward。IRL 希望恢复这个 reward，然后再用 RL 学策略。

适合场景：

- 专家动作可观察。
- reward 很难手工设计。
- 希望学到可迁移的偏好或目标。

缺点：

- 问题不适定：多个 reward 都可能解释同一行为。
- 计算复杂。
- 实践中常被更直接的 imitation 或 preference learning 替代。

RLHF

RLHF 是 Reinforcement Learning from Human Feedback，常用于大语言模型对齐。

典型流程：

1. Pretraining：在大规模语料上预训练基础模型。
2. SFT：用人工示范数据做 supervised fine-tuning。
3. Reward Model：收集人类偏好比较，训练 reward model。
4. RL Optimization：用 PPO 等算法优化 policy，使其获得更高 reward model 分数。

偏好数据形式：

$(prompt, response_{chosen}, response_{rejected})$

Reward model 学习：

$$r_{chosen} > r_{rejected}$$

常见 pairwise loss：

$$L = -\log \sigma(r_{chosen} - r_{rejected})$$

RLHF 中的 PPO

在 RLHF 中, policy 是语言模型。动作是 token, trajectory 是生成序列。

优化目标通常包含:

- reward model 分数。
- KL penalty, 限制新模型不要偏离 reference model 太远。

形式:

$$r_{total} = r_{RM} - \beta KL(\pi \parallel \pi_{ref})$$

为什么需要 KL penalty?

- 防止模型为了骗 reward model 而偏离语言质量。
- 保持和 SFT/reference model 的行为接近。
- 提高训练稳定性。

RLHF 的常见问题

- Reward hacking: 模型找到 reward model 漏洞。
- Preference data 噪声。
- Reward model 泛化问题。
- PPO 训练不稳定。
- 对齐目标难以完全由标注偏好表达。

近年也常见不用 RL 的偏好优化方法, 例如 DPO。面试中可以简单提到:

DPO 直接从偏好数据推导监督式目标, 避免显式训练 reward model 后再跑 PPO, 工程上更简单。

本章例子: 四类高级 RL 场景

例子 1: Model-based RL 做机器人规划

机器人先学习一个 dynamics model:

$$\hat{P}(s'|s,a)$$

然后在模型里模拟不同动作序列:

动作序列 1 -> 预测会撞墙
动作序列 2 -> 预测能到目标点
动作序列 3 -> 预测路径更短但风险高

再选择预测回报最高的动作序列执行。

例子 2: Offline RL 做历史推荐日志

公司已经有大量推荐日志：

用户状态、展示内容、点击/停留/购买

Offline RL 希望只用这些历史数据学习新策略。但难点是新策略可能推荐历史日志里很少出现的内容，价值估计容易外推错误。

所以 offline RL 常需要保守约束：不要让策略偏离数据分布太远。

例子 3: Imitation Learning 做驾驶

Behavior Cloning 可以用人类驾驶数据训练：

输入：摄像头、速度、导航

输出：方向盘角度、油门、刹车

问题是模型一旦偏离人类数据分布，就可能进入训练集中没见过的状态。DAgger 的思路是让专家在模型实际访问到的状态上继续标注，补齐分布偏移。

例子 4: RLHF 做聊天模型

RLHF pipeline:

1. 用 SFT 让模型学会基本回答格式。
2. 收集同一个 prompt 下多个回答的人类偏好。
3. 训练 reward model。
4. 用 PPO 等方法优化策略，同时用 KL 限制模型偏离参考模型。

面试表达：

Model-based、Offline、Imitation、RLHF 的共同点是都在解决“真实交互成本高或风险大”的问题：要么学模型模拟，要么复用历史数据，要么模仿专家，要么用人类偏好构造 reward。

本章面试高频问题

1. Model-free 和 model-based RL 区别？

Model-free 不显式学习环境模型，直接学习 policy 或 value。Model-based 学习或使用环境 dynamics 和 reward model，并基于模型 planning 或生成数据。

2. Model-based RL 的主要问题是什么？

模型误差。学到的 dynamics model 不可能完全准确，长 horizon planning 会累积误差，导致策略利用模型漏洞而在真实环境中表现差。

3. Offline RL 为什么难？

因为只能使用固定数据集，不能通过环境探索纠错。策略可能选择数据分布之外的动作，而 Q 函数对这些动作估计不可靠，容易产生 extrapolation error。

4. Behavior Cloning 的问题？

BC 把模仿学习当监督学习，简单稳定，但有 distribution shift 和 compounding error。模型部署后遇到专家数据没覆盖的状态，错误会逐步累积。

5. RLHF 的基本流程？

先预训练语言模型，再用示范数据 SFT，然后用人类偏好训练 reward model，最后用 PPO 等 RL 算法在 KL 约束下优化模型输出。

本章练习

1. 用一句话区分 online RL 和 offline RL。
2. 解释 offline RL 中的 extrapolation error。
3. 对比 Behavior Cloning 和 RL。
4. 画出 RLHF 的四阶段流程。

[章节索引](#)

[< 上一章](#) [下一章 >](#)